

Plano de Evolução SaaS

Roadmap da plataforma

Ciclartech · Plataforma de Gestão de Ativos Assistivos

Plano de Evolução: de Sistema de Empréstimos para Plataforma SaaS Ciclartech

Versão: 1.0 **Data:** 28/07/2026 **Autor:** Engenharia de Software **Status:** Plano para aprovação — nenhum código foi alterado nesta etapa **Documento base:** docs/ESPECIFICACAO_TECNICA.md (v1.1)

0. Como este documento se relaciona com o anterior

Este documento **não substitui** a Especificação Técnica v1.1 — ele a **estende**. A v1.1 definiu corretamente:

- O domínio operacional (equipamentos, beneficiários, empréstimos, devoluções, manutenção, notificações).
- A stack (Django + DRF + HTMX/Alpine + PostgreSQL + Celery/Redis).
- O padrão arquitetural (monólito modular por apps de domínio).
- A estratégia multi-tenant (schema compartilhado + `tenant_id`, com `TenantManager` e `TenantMiddleware`).
- A identidade visual e os fluxos de UX do protótipo (`dc7b60f9-CRM_Equipamentos_Ortopedicos.dc.html`).

Nada disso muda. O que este documento adiciona é a camada de **plataforma SaaS** em torno desse núcleo: hierarquia Owner/Admin/Gestor/Funcionário, segmentos de mercado com feature flags, planos e assinaturas, área `/owner`, e o módulo de Blindagem Financeira. O domínio "equipamentos ortopédicos" se generaliza para "ativos assistivos" sem exigir reescrita, porque o modelo já era desenhado por `CategoriaEquipamento` (genérico) — a mudança é de nomenclatura de produto, não de esquema de dados.

Estado atual do repositório: apenas documentação (`docs/`) e o protótipo de design enviado — nenhuma linha de código Django foi escrita ainda. Isso é uma vantagem: podemos incorporar a visão de plataforma **desde a primeira migration**, em vez de refatorar depois.

1. Análise da Arquitetura Existente e Pontos de Reaproveitamento

Elemento já definido	Decisão desta evolução
Identidade visual do protótipo (paleta #123B37 / #155E5A / #E39A5C , tipografia Public Sans, layout sidebar + header + main)	100% preservada. Vira o design system padrão do produto (app/ shell), reaproveitado também no /owner com variação sutil de tom (para diferenciar visualmente "modo plataforma" de "modo cliente")
Fluxos de UX (wizard de empréstimo, devolução em checklist, painel rápido, leitor de QR)	Preservados integralmente. São a referência de "operação em <2min" pedida agora explicitamente
Monólito modular Django (apps por domínio)	Mantido. É exatamente o padrão que permite "uma aplicação só, menus mudam por perfil" — não haverá app Django separado por segmento nem por papel
Tenant com tenant_id em cascata + TenantManager	Mantido e é a peça central da nova hierarquia. Só é estendido com campos de segmento/plano
RBAC via Group / Permission (Administrador, Atendente, Gestor, Manutenção)	Reaproveitado com ajuste de nomenclatura: os papéis do protótipo mapeiam quase 1:1 para a hierarquia pedida agora (ver seção 3). "Atendente" → "Funcionário"; "Administrador" (do tenant) permanece; entra o novo nível Owner , que fica fora do tenant
Equipamento , EventoHistorico , FotoEquipamento , Empréstimo , Manutencao	Reaproveitados sem alteração de estrutura. EventoHistorico já é a timeline pedida agora; só precisa de mais tipo_evento (Compra, Baixa)
Assinatura física por padrão / módulo digital opcional (v1.1)	Mantido sem alteração
Notificações via WhatsApp (Celery)	Reaproveitado , agora com contadores agregados para o dashboard Owner ("quantidade total de notificações")

Conclusão da análise: a v1.1 já foi desenhada, sem saber explicitamente, no formato certo para virar uma plataforma SaaS. O trabalho aqui é **aditivo**: novas entidades (Segmento , Plano , Assinatura , FeatureFlag), um novo app Django (plataforma ou owner) e uma camada de resolução de menu/permissão — não uma reescrita.

2. Visão de Produto: Ciclartech

Ciclartech deixa de ser "um sistema de empréstimo de equipamentos ortopédicos" e passa a ser descrito como:

Plataforma de Gestão de Ativos Assistivos — controla o ciclo de vida completo de um ativo (aquisição → cadastro → QR Code → empréstimo → renovação → devolução → manutenção → baixa), servindo múltiplos tipos de organização

(prefeituras/fundos sociais, home care, locadoras, hospitais, ONGs) a partir de uma única base de código multi-tenant.

O empréstimo continua sendo o módulo mais maduro (é o que já está especificado em detalhe na v1.1), mas deixa de ser tratado como "o sistema" — passa a ser um dos módulos habilitáveis por tenant via feature flag.

3. Hierarquia e Multi-Tenancy

3.1 Owner não é um tenant

Ponto arquitetural crítico: o **Owner é da plataforma, não de um cliente**. Isso significa:

- `Usuario.tenant` é **nullable**. Quando `tenant IS NULL` e `is_platform_staff=True`, o usuário é da equipe Ciclartech.
- O `TenantManager` (que filtra automaticamente toda query por `tenant_id`) **nunca** é usado nas views de `/owner` — lá as consultas são intencionalmente cross-tenant (agregações). Para isso, criamos um segundo manager, `PlatformManager` (`objects_all` ou equivalente), usado **exclusivamente** dentro do app `owner`, nunca em views de tenant. Essa separação de managers é a barreira técnica que impede um bug de "vazar" dados entre clientes: por padrão (`Model.objects`), a query é sempre restrita; o acesso cross-tenant exige uma chamada explícita e auditável.
- Nenhuma view do namespace `/app/*` (o produto para os clientes) pode importar `PlatformManager`. Isso é reforçado por um teste de arquitetura automatizado (import-linter ou teste customizado que varre `app/` procurando o símbolo proibido) — evita que isso vire regra "de boa vontade".

3.2 Hierarquia dentro do tenant

```
Owner (Ciclartech · fora do tenant)
├── Admin (dono da conta do cliente)
│   ├── Gestor (operação)
│   │   └── Funcionário (execução)
```

Implementação: reaproveita `django.contrib.auth.Group` (já decidido na v1.1), com 4 grupos fixos por tenant (`Admin`, `Gestor`, `Funcionário`, mais o grupo especial `Manutenção` do protótipo original, que pode coexistir como um papel transversal — ver nota abaixo) e um campo `nivel_hierarquico` (inteiro) no model `Papel / Perfil`, usado para: - Regra de negócio "um usuário só pode gerenciar usuários de nível hierárquico igual ou inferior ao seu" (Admin pode criar Gestor/Funcionário; Gestor não cria Admin). - Ordenação/exibição no painel de usuários.

Nota de compatibilidade: o papel "Manutenção" do protótipo v1.1 não desaparece — ele passa a ser modelado como uma **permissão adicional** (`pode_gerenciar_manutencao`) que pode ser concedida a um Funcionário ou Gestor, em vez de um 5º nível hierárquico. Isso simplifica a árvore de decisão do RBAC sem perder a capacidade descrita antes.

3.3 Matriz de permissões (reconciliada)

Ação	Owner	Admin	Gestor	Funcionário
Administrar plataforma (<code>/owner</code>)	✓	—	—	—
Cadastrar usuários/gestores/ funcionários do tenant	—	✓	—	—
Cadastrar equipamentos, categorias, unidades, fornecedores	—	✓	visualizar	visualizar
Configurar notificações, QR, feature flags do próprio tenant (dentro do plano contratado)	—	✓	—	—
Dashboard/indicadores	visão agregada de todos os tenants	✓	✓	—
Empréstimo / Devolução / Renovação / Fotos / QR	—	✓	✓ (+ aprovações)	✓ (fluxo principal)
Aprovar movimentações sensíveis (ex.: renovação fora da política)	—	✓	✓	—
Ver dados de outro tenant	✓ (com trilha de auditoria)	✗ nunca	✗ nunca	✗ nunca

4. Segmentos e Feature Flags

4.1 Modelo de dados

- **Segmento:** `id`, `codigo` (`fundo_social`, `home_care`, `locadora`, `hospital`, `ong`, ...), `nome`. Um Segmento é metadado do Tenant, não do usuário.
- **Modulo:** catálogo fixo de módulos do produto (`equipamentos`, `beneficiarios`, `emprestimos`, `renovacoes`, `manutencao`, `inventario`, `qrcode`, `relatorios`, `notificacoes`, `agenda`, `ordens_servico`, `pacientes`, `contratos`, `financeiro_basico`, `blindagem_financeira`, ...).
- **SegmentoModulo:** define o comportamento **padrão** de cada módulo por segmento (`habilitado_default`: `bool`). É a tabela que materializa a tabela de exemplo do briefing (Fundo Social: ✓ beneficiários, ✗ financeiro).

- **TenantFeatureFlag**: override pontual por tenant (`tenant` , `modulo` , `habilitado`) — permite ligar/desligar um módulo específico para um cliente sem mudar o segmento inteiro (ex.: uma prefeitura que também quer testar Blindagem Financeira).
- Resolução: `tem_modulo(tenant, "financeiro_basico")` → checa `TenantFeatureFlag` (override) → senão cai para `SegmentoModulo` do segmento do tenant → senão `False` . Resultado cacheado em Redis por tenant (invalidado ao salvar flag), pois essa checagem roda em **toda navegação de menu**.

4.2 Onde isso aparece na UI

- O menu lateral (já existente no protótipo, `navGroups`) passa a ser **gerado dinamicamente** cruzando: (a) módulos habilitados para o tenant e (b) permissões do papel do usuário. O componente visual não muda — o dado que o alimenta passa a vir de `tem_modulo()` + `has_perm()` em vez de ser estático.
- Dashboards por segmento (Fundo Social/Prefeitura, Home Care, Locadora) são **variações de composição** da mesma tela de Dashboard: os mesmos cards/gráficos (já no protótipo) são selecionados por config de segmento, não são 3 páginas de dashboard diferentes. Isso evita duplicar templates e mantém "uma aplicação só" (requisito explícito do briefing).

5. Planos e Assinaturas

5.1 Modelo de dados

- **Plano**: `segmento` (FK), `codigo` , `nome` , `limite_equipamentos` , `limite_usuarios` , `limite_unidades` , `limite_notificacoes_mes` , `limite_armazenamento_mb` , `modulos_inclusos` (M2M com `Modulo` , refina o `SegmentoModulo` por plano dentro do mesmo segmento — ex.: "Essencial" do Fundo Social não inclui Relatórios avançados, "Gestão Plena" inclui).
- **Assinatura**: `tenant` (FK, 1:1 ativo por vez), `plano` (FK), `status` (`trial` / `ativo` / `inadimplente` / `cancelado`), `ciclo` (`mensal` / `anual`), `data_inicio` , `data_fim_trial` , `data_cancelamento` , `valor_mrr` , `valor_setup` .
- **HistoricoAssinatura**: trilha de upgrades/downgrades/cancelamentos (alimenta métricas de churn no `/owner`).

5.2 Enforcement de limites

- Um `LimiteService` (camada de `services.py` , seguindo o padrão já definido na v1.1) é chamado nos pontos de criação (`criar_equipamento` , `criar_usuario` , `criar_unidade`) e valida contra `Assinatura.plano` . Ao atingir o limite, a UI mostra um estado de bloqueio "amigável" (não um erro 500) com CTA para upgrade — isso é responsabilidade da camada de apresentação, a regra vive no service.
- **Importante, alinhado ao briefing**: o sistema **nunca gera cobrança automática**. `Assinatura` e `Plano` existem para **enforcement de limites e para os indicadores do Owner**, não para processar pagamento. Integração com gateway de pagamento

(Stripe/Iugu/Vindi) fica como ponto de extensão documentado, fora do escopo desta fase.

6. Área `/owner`

`/owner` é um **namespace Django separado** (`urls.py` próprio, `owner/` app), protegido por `is_platform_staff=True` (nunca por `Group` de tenant — são sistemas de permissão diferentes, o que evita que uma escalação de privilégio dentro de um tenant chegue à área de plataforma).

6.1 Dashboard Owner — fontes de dado (reaproveitando o que já existe)

Métrica pedida	Origem do dado
Clientes ativos / em implantação / em teste	<code>Assinatura.status</code> agregada por <code>Tenant</code>
MRR / ARR / Receita de setup / Ticket médio	Agregações sobre <code>Assinatura.valor_mrr / valor_setup</code>
Clientes por segmento / por plano	<code>GROUP BY</code> em <code>Tenant.segmento / Assinatura.plano</code>
Clientes cancelados / Churn	<code>HistoricoAssinatura</code> (eventos de cancelamento / total ativo no início do período)
Totais de equipamentos/ beneficiários/usuários/ empréstimos/QR Codes/ notificações	Contagens cross-tenant sobre as tabelas já existentes (<code>Equipamento</code> , <code>Beneficiario</code> , <code>Usuario</code> , <code>Emprestimo</code> , <code>qr_code_token</code> , <code>NotificacaoEnviada</code>) via <code>PlatformManager</code>
Quantidade por tipo de cliente (municípios/hospitais/home care/ locadoras)	<code>COUNT</code> por <code>Tenant.segmento</code>
Mapa do Brasil / crescimento / novos clientes / implantações	<code>Tenant.criado_em</code> + campo de UF/município (já previsto em <code>Beneficiario / Instituicao</code> na v1.1, elevado para <code>Tenant</code>)
Saúde da plataforma / Logs	Integração com a stack de observabilidade já definida na v1.1 (Sentry, logs estruturados) — o <code>/owner</code> apenas exibe esses dados, não substitui a stack de monitoramento
Feature Flags / Planos / Assinaturas / Billing	CRUD sobre os models da seção 4 e 5

Nenhuma dessas métricas exige um novo motor de analytics nesta fase — são agregações SQL sobre tabelas que já existiam ou que estamos adicionando de forma incremental. Um data warehouse separado só se justifica se/quando o volume de tenants chegar na casa dos milhares e as agregações em tempo real começarem a pesar no banco transacional (ponto de atenção para uma fase futura, não agora).

7. Dashboards por Segmento

Reaproveitando a mesma tela de Dashboard do protótipo (grid de cards + gráfico de barras + lista), variando a **composição de widgets** por segmento:

- **Fundo Social / Prefeitura:** exatamente os widgets já prototipados na v1.1 (disponíveis, emprestados, manutenção, atrasados, unidades, bairros).
- **Home Care:** widgets novos, mas reaproveitando os mesmos componentes visuais (card , lista com badge , timeline) — Pacientes (renomeação de `Beneficiario` na camada de apresentação, sem duplicar model), Ordens de Serviço (novo model `OrdemServico` , análogo em estrutura a `Emprestimo` + `Manutencao`), Agenda/Visitas Técnicas (reaproveita o model `Agenda` já esboçado na v1.1), Entregas/Retiradas (reaproveita `Emprestimo` / `Devolucao` renomeados na UI).
- **Locadora:** Contratos (novo model `Contrato` , com `Cliente` podendo reaproveitar a estrutura de `Beneficiario` generalizada para "Parte Contratante"), Receita e indicadores financeiros (módulo Blindagem Financeira, seção 8).

Ponto de atenção arquitetural: **Beneficiario está sendo generalizado** conceitualmente para "a pessoa/entidade para quem o ativo é destinado" (beneficiário social, paciente, cliente locatário). Recomendação: **não** criar 3 models distintos agora. Em vez disso, o model ganha um campo `tipo_relacao` (`beneficiario` / `paciente` / `cliente`) usado só para rótulo e regras finas de formulário — a estrutura de dados (endereço, contato, documentos, histórico de empréstimos) é a mesma nos três casos. Isso evita explosão de tabelas e mantém o núcleo (`Emprestimo`) único entre segmentos.

8. Módulo Blindagem Financeira

- Disponível apenas quando `tem_modulo(tenant, "blindagem_financeira")` é verdadeiro (por padrão, só para segmentos `locadora` e `home_care` , via `SegmentoModulo`).
- Novo campo em `Equipamento` : `valor_referencia` (decimal, opcional — valor de mercado/reposição do ativo). Sem esse dado o módulo simplesmente não é habilitado (o próprio enforcement de feature flag cobre isso; não precisamos de uma migration obrigatória em tenants que não usam o módulo).
- Indicadores calculados (somente leitura, **sem geração de cobrança automática** — restrição explícita do briefing, reforçada aqui):
- **Valor do patrimônio em circulação** = soma de `valor_referencia` de equipamentos com `status='emprestado'` .
- **Valor exposto** = soma de `valor_referencia` de equipamentos com `status='emprestado'` e `Emprestimo` em atraso.
- **Equipamentos em atraso** = reaproveita a mesma consulta já usada na Agenda/Dashboard (RF014 da v1.1).
- **Valor potencial** = soma de `valor_referencia` de equipamentos `disponivel` (capacidade ociosa de geração de receita, para Locadora).

- Tecnicamente é um `selector.py` novo dentro de um app `financeiro` (ou dentro de `relatorios`, a decidir na Fase 5), sem necessidade de novo motor de regras — é composição de queries sobre entidades que já existem.

9. Rotas

Confirmação da diretriz do briefing — **uma única aplicação Django**, dois namespaces de URL:

```
/owner/...           → equipe Ciclartech (is_platform_staff)
/app/dashboard
/app/equipamentos
/app/beneficiarios    (rótulo dinâmico: Beneficiários | Pacientes | Clientes, por
segmento)
/app/emprestimos
/app/devolucoes
/app/renovacoes
/app/manutencao
/app/inventario
/app/qrcode
/app/usuarios
/app/unidades
/app/relatorios
/app/configuracoes
```

Cada view sob `/app/*` é protegida por (1) `TenantMiddleware` (isolamento — já existente na v1.1), (2) checagem de feature flag do módulo, (3) permissão do papel (`Group / nivel_hierarquico`). O menu lateral é a **projeção visual** dessas três checagens — não existe rota "escondida" que dependa só do menu não mostrar o link; toda view revalida no servidor (defesa em profundidade, mesmo princípio já aplicado ao isolamento multi-tenant na v1.1).

10. QR Code, Fotos e Timeline — confirmação de reaproveitamento

- **QR Code:** o fluxo "Escanear → Abrir ficha → Executar ação" já é exatamente o que está prototipado (modal de QR → `qrViewEquip / qrStartReturn`). Nenhuma mudança estrutural — apenas garantir que, em mobile, o botão de QR seja o ponto de entrada primário (já é: FAB + botão no header mobile no protótipo).
- **Fotos (comparação entrega x devolução):** já modelado na v1.1 (`FotoEmprestimo` com `momento=retirada/devolucao` , e a tela de comparação lado a lado já existe no protótipo, aba "Fotos" da ficha do equipamento). Sem mudanças.
- **Timeline:** `EventoHistorico` já cobre isso; adicionar os `tipo_evento` "Compra" e "Baixa" (hoje o protótipo já tem "Baixado" como status, falta o evento correspondente

na timeline) para fechar o ciclo completo pedido agora (Compra → Cadastro → Empréstimos → Renovações → Manutenções → Devoluções → Baixa).

11. Responsividade Mobile-First

A v1.1 já previa `isMobile` com layout adaptado (bottom nav, FAB, painel deslizante). Para esta evolução:

- Reforçar como **princípio de aceite**, não só de layout: os fluxos de Emprestar/ Devolver/Renovar/Escanear QR/Fotos devem ser testados explicitamente em viewport mobile antes de qualquer PR ser considerado concluído (critério objetivo: empréstimo completo em <2min, devolução em <1min, medido em teste manual cronometrado — vira parte do checklist de Definition of Done).
- Tecnicamente, HTMX + Alpine já são mobile-friendly por não dependerem de bundle pesado de SPA; adicionar, em fase futura, um **manifest PWA leve** (ícone, tela cheia, cache do shell) para reduzir fricção de "abrir o navegador" no dia a dia do funcionário — não é bloqueante para o MVP.

12. Plano de Implementação em Fases

Princípio geral: cada fase entrega algo demonstrável e não bloqueia a próxima. A ordem prioriza primeiro ter **um tenant de referência funcionando de ponta a ponta** (o que a v1.1 já especificou em detalhe) antes de multiplicar segmentos e adicionar a camada comercial (planos/billing/Owner) — isso é deliberado: **não vale a pena investir em métricas de MRR/churn antes de o produto operacional estar validado com um cliente real.**

Fase 0 — Fundação técnica e multi-tenant real

Objetivo: esqueleto Django rodando com isolamento de tenant validado por teste automatizado, antes de qualquer tela de negócio. - Projeto Django + apps (`core` , `contas` , `equipamentos` , `beneficiarios` , `emprestimos` , `manutencao` , `notificacoes` , `owner`). - `Tenant` , `Usuario` (com `tenant nullable + is_platform_staff`), `TenantMiddleware` , `TenantManager` . - Grupos `Admin` / `Gestor` / `Funcionário` + campo `nivel_hierarquico` . - Suíte de testes de isolamento multi-tenant (RNF017) — **critério de saída da fase:** nenhum teste de "vazamento entre tenants" falha. - CI (lint + testes) no GitHub Actions.

Fase 1 — MVP operacional (tenant único, segmento Fundo Social/ Prefeitura)

Objetivo: entregar o que a v1.1 já especificou em detalhe, validando com um cliente piloto real. - Cadastro de equipamentos + fotos + QR Code + categorias. - Cadastro de

beneficiários + documentos. - Wizard de empréstimo (assinatura física por padrão), devolução, renovação. - Manutenção, timeline (`EventoHistorico`), dashboard operacional, agenda. - Notificações WhatsApp (Celery + Celery Beat). - Interface mobile-first completa para o fluxo Funcionário. - **Critério de saída:** um tenant real opera o ciclo completo empréstimo→devolução sem suporte manual da engenharia.

Fase 2 — Camada de segmentação e feature flags

Objetivo: provar que o mesmo código atende um segundo tipo de cliente sem fork. - Models `Segmento` , `Modulo` , `SegmentoModulo` , `TenantFeatureFlag` + serviço `tem_modulo()` com cache. - Menu lateral dinâmico (módulos + permissões). - Generalização de rótulo de `Beneficiario` (`tipo_relacao`) e ativação dos widgets de dashboard por segmento (Home Care como segundo segmento-piloto, por ser o que mais reaproveita do que já existe). - **Critério de saída:** dois tenants de segmentos diferentes ativos, cada um vendo só seus módulos.

Fase 3 — Planos, limites e assinaturas

Objetivo: viabilizar comercialmente múltiplos tenants sem intervenção manual de engenharia a cada novo cliente. - Models `Plano` , `Assinatura` , `HistoricoAssinatura` . - `LimiteService` com enforcement de limites por plano nos pontos de criação. - Onboarding self-service de tenant em modo trial (criação de `Tenant` + `Assinatura(status=trial)` + usuário Admin inicial). - **Critério de saída:** um novo tenant pode ser criado e operar em modo trial sem alteração de código ou deploy.

Fase 4 — Área `/owner`

Objetivo: dar visibilidade de negócio à Ciclartech sobre a base de clientes. - App `owner` com `PlatformManager` isolado (barreira técnica descrita na seção 3.1) e teste de arquitetura que impede uso fora do namespace. - Dashboard Owner completo (seção 6.1): métricas de negócio, totais operacionais agregados, mapa do Brasil, gráfico de crescimento. - CRUD de Planos/Feature Flags/Assinaturas via `/owner` (hoje gerido só via Django Admin cru). - Painel de saúde da plataforma (agregando o que a stack de observabilidade da v1.1 já produz). - **Critério de saída:** Owner consegue responder "quantos clientes ativos, MRR atual e churn do mês" sem consultar o banco diretamente.

Fase 5 — Locadora e Módulo Blindagem Financeira

Objetivo: terceiro segmento, o mais distante do núcleo original, prova final de que a arquitetura generaliza. - Model `Contrato` , dashboard Locadora (contratos, receita, equipamentos alugados/disponíveis/atrasados). - Campo `valor_referencia` em `Equipamento` (opt-in por tenant/segmento). - Módulo `blindagem_financeira` (indicadores de patrimônio em circulação/exposto/potencial), **sem geração de cobrança automática** — reforço explícito do requisito. - **Critério de saída:** tenant do segmento Locadora opera com indicadores financeiros corretos, validado com dados reais de um piloto.

Fase 6 — Hardening, escala e observabilidade avançada

Objetivo: preparar para "milhares de tenants", não apenas dezenas. - Revisão de índices (todo `unique_together` /índice composto por `tenant_id` auditado sob carga). - Cache de feature flags e de agregações do `/owner` (evitar que o dashboard de plataforma penalize o banco transacional). - Reavaliação, com dados reais de volume, de migrar tenants de maior porte para isolamento por schema (`django-tenants`) — decisão já deixada em aberto na v1.1 (seção 3.3), revisitada aqui com dados reais em vez de estimativa. - Auditoria de segurança completa (RNF006/RNF007/RNF011), testes de penetração básicos no isolamento multi-tenant. - PWA leve para o fluxo mobile (manifest + cache do shell).

13. O que explicitamente não muda nesta evolução

Para deixar claro o que foi preservado, conforme instruído:

- Paleta de cores, tipografia, componentes visuais (cards, badges, wizard, painel lateral) do protótipo — **inalterados**.
 - Fluxos de UX já validados (empréstimo, devolução, QR) — **inalterados** em sua sequência de passos.
 - Stack tecnológica definida na v1.1 (Django, DRF, HTMX/Alpine, PostgreSQL, Celery/Redis, S3) — **inalterada**.
 - Padrão arquitetural monólito modular — **reforçado**, não substituído.
 - Estratégia multi-tenant por `tenant_id` em schema compartilhado — **reaproveitada como base de toda a hierarquia Owner/Admin/Gestor/Funcionário**.
 - Assinatura física por padrão / assinatura digital como módulo opcional — **inalterado**.
-

14. Próximos Passos Imediatos

1. Validar com o solicitante a nomenclatura final de segmentos e módulos (lista fechada de `Modulo` antes de codificar `SegmentoModulo` , para evitar migrations de catálogo repetidas).
2. Aprovar este plano de fases (ou reordenar prioridades — ex.: adiantar `/owner` se houver pressão comercial para demonstrar a métrica de MRR antes do piloto operacional).
3. Iniciar Fase 0 (fundação técnica) somente após aprovação explícita, conforme solicitado ("antes de começar a modificar o código").